

THE bCCNN AS A WHOLE

What it estimates, why early stopping regularises it, how its reserve is judged and how its uncertainty can be measured

Study notes on the bCCNN strand of the research assignment

Prepared with the generative artificial intelligence tool Claude (Anthropic)
as study material for the BComHons research assignment
Department of Statistics and Actuarial Science
Stellenbosch University

Not for submission for assessment

October 2026

Note on the preparation of these notes

These notes were prepared with the generative artificial intelligence (AI) tool Claude (Anthropic) at the request of the student, as study material on the blended cross-classified neural network (bCCNN) strand of the research assignment. They explain the mathematics of the model, the code in the student's repository (Masters-Research-R) and the literature that the code follows.

The notes are not the student's original work and are not intended for submission for assessment. Section 2.6.3 of the Research Assignment Guide (Department of Statistics and Actuarial Science, 2026: 7-8) prohibits the use of AI tools to generate content that is submitted for assessment and requires a declaration of any use of such tools. For this reason the plagiarism declaration and the declaration on the use of artificial intelligence tools of the Department template are not included. The layout follows the technical requirements in Chapter 5 of the Guide and the Department template.

Abstract

This report joins three companion explainers on the blended cross-classified neural network, a claims reserving model that starts at the chain ladder and corrects it with a small neural network trained by gradient descent. It addresses the questions that sit above the individual pieces: which quantity the model estimates and why the Poisson deviance is the right criterion for it, why a network with more parameters than observed cells does not reproduce the triangle when it is stopped early, how the resulting reserve is judged in the code and in the literature, and how its uncertainty can be measured.

The model estimates the expected outstanding payments. Early stopping from the chain-ladder start acts approximately as a ridge penalty towards the chain ladder whose strength is inversely proportional to the number of steps, and the step-size bound of the optimiser limits how far the correction can move. The reserve is judged by its back-test bias, by the out-of-sample deviance and by an out-of-time test error; the scores that need a standard error require an uncertainty measure that the code does not yet compute for the network. The conditional mean squared error of prediction can be estimated from the process variance under the over-dispersed Poisson assumption, a parametric bootstrap of the whole procedure and a seed study whose average is the nagging predictor.

A synthetic illustration shows a correction of a few per cent, a bootstrap estimation variance equal to that of the chain ladder, a modest seed spread and a large bias that both models share and that only the back-test reveals.

Key words: Claims reserving; Chain ladder; Over-dispersed Poisson model; Neural networks; Early stopping; Regularisation; Mean squared error of prediction; Bootstrap; Nagging predictor; Scoring rules

Table of contents

Note on the preparation of these notes.....	ii
Abstract.....	iii
List of tables.....	vi
List of figures.....	vii
List of abbreviations and/or acronyms.....	viii
CHAPTER 1 INTRODUCTION.....	1
1.1 INTRODUCTION.....	1
1.2 THE BCCNN IN ONE FORMULA.....	2
1.3 SCOPE, LIMITATIONS AND APPROACH.....	2
1.4 OUTLINE OF THE REPORT.....	3
CHAPTER 2 WHAT THE bCCNN ESTIMATES.....	4
2.1 INTRODUCTION.....	4
2.2 THE TARGET IS THE MEAN OF EACH CELL.....	4
2.3 BALANCE: A PROPERTY THE CHAIN LADDER HAS AND THE BCCNN LOSES.....	5
2.4 WHAT THE CORRECTION CAN AND CANNOT REPRESENT.....	5
2.5 SUMMARY.....	6
CHAPTER 3 REGULARISATION BY EARLY STOPPING.....	7
3.1 INTRODUCTION.....	7
3.2 THE BUDGET ON THE PARAMETERS.....	7
3.3 EARLY STOPPING AS SHRINKAGE TOWARDS THE CHAIN LADDER.....	8
3.4 DROPOUT.....	8
3.5 SUMMARY.....	9
CHAPTER 4 EVALUATION OF THE RESERVE.....	10
4.1 INTRODUCTION.....	10
4.2 THE BACK-TEST AGAINST THE SIMULATED TRUTH.....	10
4.3 THE OUT-OF-TIME TEST ERROR.....	10
4.4 THE CLASSICAL FITS AND THE SCORES IN R/SCORING.....	11

4.5	THE THREE DIAGNOSTIC PLOTS.....	11
4.6	SUMMARY.....	11
CHAPTER 5 UNCERTAINTY OF THE RESERVE.....		12
5.1	INTRODUCTION.....	12
5.2	THE DECOMPOSITION.....	12
5.3	PROCESS VARIANCE.....	12
5.4	ESTIMATION VARIANCE: THE PARAMETRIC BOOTSTRAP.....	13
5.5	SEED VARIANCE AND THE NAGGING PREDICTOR.....	13
5.6	PUTTING THE NUMBERS TOGETHER.....	14
5.7	SUMMARY.....	14
CHAPTER 6 THE bCCNN IN CONTEXT AND A NUMERICAL ILLUSTRATION.....		15
6.1	INTRODUCTION.....	15
6.2	THE BCCNN AMONG THE MODELS OF THE THESIS.....	15
6.3	WHERE THE PIECES ARE IN THE CODE.....	15
6.4	NUMERICAL ILLUSTRATION.....	16
6.5	SUMMARY AND CONCLUSIONS.....	18
References.....		20
APPENDIX A LIST OF SYMBOLS.....		22

List of tables

Table 1.1 The four explainers.....	1
Table 6.1 Where the bCCNN sits among the models of the thesis.....	15
Table 6.2 Where the formulas and steps of this report are in the code.....	16
Table 6.3 Quantities of early stopping and uncertainty on the synthetic run (amounts in millions of the synthetic data).....	17
Table A.1 Symbols used in this report.....	22

List of figures

Figure 6.1 The learned correction of the synthetic refit and the interaction built into the synthetic data 17

List of abbreviations and/or acronyms

bCCNN	Blended cross-classified neural network
ccODP	Cross-classified over-dispersed Poisson
CRPS	Continuous ranked probability score
EV	Estimation variance
GLM	Generalised linear model
IWLS	Iteratively weighted least squares
LoB	Line of business
MLE	Maximum likelihood estimate
MSEP	Mean squared error of prediction
ODP	Over-dispersed Poisson
PIT	Probability integral transform
PV	Process variance
RMSprop	Root mean square propagation

CHAPTER 1

INTRODUCTION

1.1 INTRODUCTION

The blended cross-classified neural network (bCCNN) of Gabrielli, Richman and Wüthrich (2020), referred to as Paper C as in the code, embeds the cross-classified over-dispersed Poisson (ccODP) model, whose reserve is the chain-ladder reserve, in a feed-forward neural network and lets the network correct it in a limited number of gradient steps. Three companion explainers each take one piece of the bCCNN apart: companion I, *The mathematics behind the bCCNN*, treats the ccODP model, the network and its chain-ladder start; companion II, *The fitting procedure and training of the bCCNN*, treats the data, the optimiser, early stopping and the outputs; and companion IV, *The time-aware split for the bCCNN*, treats the rolling-origin partitions. This report, companion III, puts the pieces back together.

The question addressed is the one that sits above the companions, and it has four parts: what quantity the bCCNN estimates and why the Poisson deviance is the right criterion for it; why a network with 547 parameters can be fitted to 210 cells without reproducing them, that is, what early stopping from the chain-ladder start does mathematically; how the resulting reserve is judged in the code and in the papers; and how its uncertainty can be measured, which the code does not yet do for the bCCNN but does for the classical models. Table 1.1 is the map of the four explainers.

Table 1.1 The four explainers

Question	Where it is answered
What is the ccODP, why is it the chain ladder, what is the network, what happens at the start	Companion I
Which data train, which data stop the training, what the optimiser does, what is random, what the outputs are	Companion II
What the whole thing estimates, why it does not overfit, how it is judged, how uncertain it is	Companion III (this report)
How the time-aware partitions are built, what they guarantee, how they score the procedure out of time	Companion IV

The report reads the functions `bccnn_calibrate()` and `bccnn_tables()`, the files `R/metrics.R` and `R/scoring`, and the classical wrappers in `R/classical_models.R` against Paper C, Härkönen (2021), Wüthrich and Merz (2019), whose combined actuarial neural network idea the bCCNN follows, and England and Verrall (2002). The classical models it compares with are the chain ladder, the distribution-free model of Mack (1993) and the generalised linear model (GLM) form of the ccODP.

1.2 THE BCCNN IN ONE FORMULA

Putting companions I and II together, the reported model is the chain ladder with a multiplicative correction learned in t^* gradient steps of root mean square propagation (RMSprop):

$$\begin{aligned} \log \mu_{i,j}(\hat{\theta}) &= \underbrace{\hat{c} + \hat{\alpha}_i + \hat{\beta}_j}_{\log \hat{\mu}_{i,j}^{cc}, \text{ the chain ladder}} + \underbrace{\Gamma_{\hat{\theta}}(i,j)}_{\text{learned in } t^* \text{ steps}}, \\ \Gamma_{\theta}(i,j) &= (w-1)(\hat{\alpha}_i + \hat{\beta}_j) + (c - \hat{c}) + \langle B, z^{(3)}(i,j) \rangle, \\ \Gamma_{\theta_0} &\equiv 0, \end{aligned} \quad (1.1)$$

and the reserve is

$$\begin{aligned} \hat{R}^{bCCNN} &= \sum_{(i,j) \in F} \hat{\mu}_{i,j}^{cc} e^{\Gamma_{\hat{\theta}}(i,j)}, \\ \hat{R}^{cc} &= \sum_{(i,j) \in F} \hat{\mu}_{i,j}^{cc}. \end{aligned} \quad (1.2)$$

Here $\hat{c}$, $\hat{\alpha}_i$ and $\hat{\beta}_j$ are the maximum likelihood estimates (MLE) of the ccODP intercept, accident-period effects and development-period effects, $\hat{\mu}_{i,j}^{cc}$ are the fitted ccODP means, (w, c, B) are the weights and the bias of the output neuron, $z^{(3)}(i,j)$ is the last hidden layer, θ_0 is the start, $\hat{\theta}$ the fitted parameters and F the set of future cells; all symbols are collected in the List of Symbols at the end of this report.

Everything about the bCCNN is a statement about Γ : it is zero at the start (companion I, Property 1 in Section 3.7); its first step is a linear combination of ten random features of $(\hat{\alpha}_i, \hat{\beta}_j)$ (companion I, Property 2 in Section 4.5); it is a smooth function of the two scalars $\hat{\alpha}_i$ and $\hat{\beta}_j$ only (companion I, Section 3.5); its size is bounded by the number of steps (Chapter 3); and it is evaluated, for the reserve, at pairs $(\hat{\alpha}_i, \hat{\beta}_j)$ with $i+j > n+1$ that never occur in the training cells. The relative-difference heat map of the script is $e^{\Gamma} - 1$.

1.3 SCOPE, LIMITATIONS AND APPROACH

The report is explanatory: it derives and interprets formulas and does not report results on real data. The numbers of Chapter 6 come from a synthetic illustration only, the same 20×20 triangle as in the numerical check of companion II, and they say nothing about the simulated portfolio of the thesis. They were computed with the Python check script `bccnn_check.py`, which re-implements the ccODP fit, the network, the Keras version of RMSprop and dropout in numpy and is kept in the repository folder `Claude outputs/bccnn-explainers`; no Keras or R run was made for this report. The bootstrap, the seed study and the balance correction discussed below are not part of the code and are presented as extensions of it.

The quotations of Paper C are taken through the literature notes in the repository (thesis/Hyperparameter search), which quote the working-paper version of Gabrielli, Richman and Wüthrich (2018) and the version in the doctoral thesis of Gabrielli (2020b); these notes have not been re-

checked against the published article. The quotations of Al-Mudafer, Avanzi, Taylor and Wong (2021) are taken from the same notes. The quotations of Härkönen (2021) were checked against the text of the thesis and are cited by its printed page numbers.

The mathematics was verified in two ways. The derivations (the strict consistency of the deviance, the step-size bound, the quadratic picture of early stopping and the variance decomposition) are written out in full with their textbook sources. The bound on the parameters, the size of the learned correction, the process variance, a parametric bootstrap and a seed study were then evaluated on the synthetic run, so that each formula can be compared with a realised number (Table 6.3). The quadratic picture of Chapter 3 is an approximation for plain gradient descent and holds for RMSprop only in the per-coordinate sense stated there.

1.4 OUTLINE OF THE REPORT

Chapter 2 establishes what the bCCNN estimates: the mean of each cell, a target for which the Poisson deviance is strictly consistent, and it discusses the balance property and what the correction can and cannot represent. Chapter 3 explains why 547 parameters on 210 cells do not over-fit: the step-size budget, early stopping as shrinkage towards the chain ladder, and dropout. Chapter 4 describes how the reserve is judged: the back-test, the out-of-time test error, the classical fits and scores, and the diagnostic plots. Chapter 5 treats the uncertainty of the reserve: the decomposition of the mean squared error of prediction, the process variance, a parametric bootstrap and the seed variance. Chapter 6 places the bCCNN among the models of the thesis, maps the formulas to the code, gives the numerical illustration and closes with a summary and conclusions.

CHAPTER 2

WHAT THE bCCNN ESTIMATES

2.1 INTRODUCTION

Chapter 1 wrote the bCCNN as the chain ladder multiplied by a learned correction e^T . This chapter asks what that formula estimates. It shows that the target is the mean of each cell and that the Poisson deviance is the right criterion for it, that the bCCNN loses the balance property of the chain ladder, and which departures from the chain ladder the correction can and cannot represent.

2.2 THE TARGET IS THE MEAN OF EACH CELL

The ccODP assumption (companion I, (2.1)) and the network both model the *expected* payment of a cell, $E[Y_{i,j}]$, with the cells independent. The reserve (1.2) is therefore an estimate of the expected outstanding payments, $E[R] = \sum_F E[Y_{i,j}]$, the actuarial best estimate. Nothing in the bCCNN estimates a distribution, a quantile or a variance; that is why its dispersion needs the separate rule of companion I, Section 5.3, and its uncertainty the methods of Chapter 5.

The Poisson deviance is the right training, validation and test criterion for this target, because it is strictly consistent for the mean. For one cell, with $d(y, \mu) = \mu - y + y \log(y/\mu)$,

$$\begin{aligned} \frac{\partial}{\partial \mu} E[d(Y, \mu)] &= \frac{\partial}{\partial \mu} (\mu - E[Y] \log \mu + E[Y \log Y] - E[Y]) \\ &= 1 - \frac{E[Y]}{\mu}, \end{aligned} \tag{2.1}$$

which vanishes only at $\mu = E[Y]$, where the expected loss is minimal (the second derivative $E[Y]/\mu^2$ is positive). This holds for *any* distribution of Y with that mean, not only an over-dispersed Poisson (Gneiting, 2011; Wüthrich and Merz, 2023: Chapter 4, where the Poisson deviance is the Bregman divergence of $y \log y$). Hence minimising the in-sample deviance aims the network at the cell means; choosing t^* by the validation deviance selects the step whose means are closest, in expectation, to the means of the validation cells; and the out-of-sample deviance on the true lower triangle judges the means where they matter. The reserve bias, $\hat{R} - R$, is the same comparison summed over the future cells, and a mean-consistent fit of every cell gives an unbiased reserve, while the converse is not true (cell errors can cancel in the sum). This is why the tables report both.

2.3 BALANCE: A PROPERTY THE CHAIN LADDER HAS AND THE BCCNN LOSES

The ccODP MLE reproduces the observed row and column totals (companion I, (2.9)); in particular the fitted total over the observed triangle equals the observed total, $\sum_o \hat{\mu}_{i,j}^{cc} = \sum_o y_{i,j}$. This 'balance property' (Wüthrich and Merz, 2023: Section 5.1.5; for networks, Wüthrich, 2020) is lost after the first gradient step, because $\Gamma \neq 0$ breaks the marginal-total equations: the fitted total of the bCCNN over O is close to, but not equal to, the observed total, and the gap is a bias of the fitted level that propagates into the reserve. The code does not correct it. A one-line correction exists, rescaling the means so that the in-sample total is matched,

$$\tilde{\mu}_{i,j} = \mu_{i,j}(\hat{\theta}) \cdot \frac{\sum_o y_{i,j}}{\sum_o \mu_{i,j}(\hat{\theta})}, \quad (2.2)$$

which is the same as refitting the output intercept c by its own maximum likelihood equation with the rest of $\hat{\theta}$ held fixed; the neural network chain-ladder strand of the repository has an analogous balance correction (its sensitivity run S4). Whether to apply (2.2) to the bCCNN is a modelling decision; it is mentioned here because the in-sample total is the first thing to read off `mu_bCCNN` when judging a fit.

2.4 WHAT THE CORRECTION CAN AND CANNOT REPRESENT

Because $z^{(0)}(i, j) = (\hat{\alpha}_i, \hat{\beta}_j)$, the correction is $\Gamma(i, j) = g(\hat{\alpha}_i, \hat{\beta}_j)$ for a smooth surface g on the plane. Three structural consequences follow.

Interactions, yes. The ccODP assumes one development pattern for all accident periods. A surface over (α, β) can bend that pattern differently for different accident periods, which is exactly what the cumulative-development-factor plot of companion I, Section 5.4, displays: parallel rows for the chain ladder, diverging rows for the bCCNN. This is the effect Paper C and Härkönen report as the gain of the 'boosting'.

Calendar effects, only indirectly. A payment inflation acting along the diagonals $k = i + j - 1$ is a function of $i + j$, which g can imitate only to the extent that $\hat{\alpha}$ is monotone in i and $\hat{\beta}$ in j ; there is no calendar input. The chain ladder cannot represent it either, so both models share this blind spot, and a diagonal pattern in the Pearson residuals of *both* models is the signature to look for.

Extrapolation into the lower triangle. The training cells have $i + j \leq n + 1$: large $\hat{\alpha}_i$ (late accident periods) occur in training only with early development periods, and very negative $\hat{\beta}_j$ (late development periods) only with early accident periods. The future cells pair late accident periods with late development periods, a corner of the (α, β) plane with no training support. There g is an extrapolation of a composition of tanh functions, which saturate, so the extrapolation is bounded but not controlled by data. This is the

structural reason for keeping t^* small, for the finding of Paper C that the in-sample validation did not predict the out-of-sample gain for every line of business (LoB), and for the out-of-time test partitions of companion IV. The short tail of the simulated portfolio helps: most of the reserve sits in the first development periods of the latest accident periods, where the extrapolation is short.

2.5 SUMMARY

The bCCNN estimates the expected payment of each cell and, summed over the future cells, the expected outstanding payments. The Poisson deviance is strictly consistent for this target under any distribution with the given mean, which justifies its use for training, validation and testing, while the reserve bias is the same comparison aggregated over the future cells. Unlike the chain ladder, the bCCNN does not reproduce the observed total; the rescaling (2.2) would restore it. The correction is a smooth surface over the two ccODP effects: it can represent interactions between accident and development periods, it sees calendar effects only indirectly, and in the lower triangle it extrapolates into a region without training support. Chapter 3 explains why this surface stays close to the chain ladder.

CHAPTER 3

REGULARISATION BY EARLY STOPPING

3.1 INTRODUCTION

Chapter 2 showed that the correction Γ is a surface that must extrapolate into the lower triangle, which makes its size a central concern. This chapter answers the question why 547 parameters on 210 cells do not over-fit. It derives a hard budget on how far the parameters can move from the chain-ladder start, interprets early stopping as shrinkage towards the chain ladder, and describes dropout as the third regulariser.

3.2 THE BUDGET ON THE PARAMETERS

The count of Härkönen (2021: 23, equation (76)) and Table 3.1 of companion I agree: 547 trainable numbers, 210 cells. A network of that size can interpolate the training triangle, and a run long enough would do so; Paper C notes that "after that the algorithm starts to over-fit to the observations" (Gabrielli *et al.*, 2018: 12-13). Three things prevent it in the procedure: the start at the chain ladder, the number of steps, and dropout.

The first two combine into a hard bound. RMSprop with the rule of Keras satisfies $|g_t|/\sqrt{v_t} \leq 1/\sqrt{1-\rho}$ at every step (companion II, (3.3)), so after t steps every coordinate of θ is within

$$|\vartheta_t - \vartheta_0| \leq \frac{\eta}{\sqrt{1-\rho}} t = 0.00316 t \quad (3.1)$$

of its start. Through (1.1), with $|z^{(3)}| \leq 1$ componentwise, the log-scale correction is then bounded by

$$\begin{aligned} |\Gamma_{\theta_i}(i, j)| &\leq |w_t - 1| |\hat{\alpha}_i + \hat{\beta}_j| + |c_t - \hat{c}| + \sum_{r=1}^{10} |B_{t,r}| \\ &\leq 0.00316 t (|\hat{\alpha}_i + \hat{\beta}_j| + 11). \end{aligned} \quad (3.2)$$

For $t^* = 300$ and a cell with $|\hat{\alpha}_i + \hat{\beta}_j| \leq 5$, (3.2) gives $|\Gamma| \leq 15$, which is not a useful number: the bound is a worst case in which every coordinate moves in the same direction at the maximal rate at every step. In practice the gradients change sign and the realised distance is smaller (Section 6.4: in the synthetic run the largest move of any of the 547 coordinates after 407 steps was 0.31, a quarter of the bound 1.29, and the ten output weights B that carry the correction moved by at most 0.03). What (3.1) and (3.2) do establish is that the number of steps is the control on the distance of the model from the chain ladder, and that no run of t steps can produce a correction larger than linear in t .

3.3 EARLY STOPPING AS SHRINKAGE TOWARDS THE CHAIN LADDER

The textbook argument (Goodfellow, Bengio and Courville, 2016: Section 7.8; Bishop, 1995) makes the budget precise in a quadratic picture. Replace the loss near its minimiser θ^* (the interpolating network) by its second-order expansion $\ell(\theta) \approx \ell(\theta^*) + 1/2(\theta - \theta^*)^T H(\theta - \theta^*)$ with H positive semi-definite, eigenvalues $\lambda_r \geq 0$ and eigenvectors u_r . Plain gradient descent with learning rate η from θ_0 gives, in the eigen-coordinates $\xi_r = u_r^T(\theta - \theta_0)$ and $\xi_r^* = u_r^T(\theta^* - \theta_0)$,

$$\xi_{t,r} = \left(1 - (1 - \eta \lambda_r)^t\right) \xi_r^*. \quad (3.3)$$

Ridge regression from the same start, $\min_{\theta} \ell(\theta) + \kappa/2 \|\theta - \theta_0\|^2$, gives $\xi_r = \frac{\lambda_r}{\lambda_r + \kappa} \xi_r^*$. The two agree when $(1 - \eta \lambda_r)^t = \kappa / (\lambda_r + \kappa)$, and for $\eta \lambda_r \ll 1$ and large t both sides are approximately $e^{-\eta \lambda_r t}$ and $1/(1 + \lambda_r/\kappa)$, so

$$\kappa \approx \frac{1}{\eta t}. \quad (3.4)$$

In words: t gradient steps from the chain-ladder start are approximately a ridge fit penalised towards the chain ladder with penalty $1/(\eta t)$. Directions of the parameter space in which the data pull strongly (large λ_r : the correction is well supported by many cells) are learned almost completely within a few hundred steps, $1 - e^{-\eta \lambda_r t} \approx 1$; directions in which the data pull weakly (small λ_r : idiosyncratic cells, the extrapolation corner of Section 2.4) are barely moved, $\eta \lambda_r t \ll 1$. The 547 parameters are therefore not 547 free degrees of freedom; the effective number is the number of eigen-directions with $\eta \lambda_r t^* \gtrsim 1$, and it grows with t^* . RMSprop rescales the learning rate of each coordinate by its running gradient size, so (3.4) holds per coordinate with an effective η of order $\eta/\sqrt{v}$, but the picture is the same. This is the precise sense in which 'the number of gradient descent steps' is the single hyperparameter that Paper C and Härkönen tune, and why $t^* = 0$ is the chain ladder and $t^* \rightarrow \infty$ the interpolating network.

3.4 DROPOUT

Dropout (companion I, Section 3.6) is the third regulariser: with 10% of the hidden units removed at random at each step, the gradient the network follows is that of a random sub-network, and features that depend on particular co-adapted units are penalised. The phrasing of Härkönen is the standard one: the neurons "have to learn to co-operate with a random sample of other neurons" (Härkönen, 2021: 12). Its cost is visible in the curves: the recorded validation deviance is noisy even though it is computed without dropout, because the parameters themselves jitter, and the exact minimum of companion II, (4.2), can land on a lucky step. In the version of Paper C in the doctoral thesis of Gabrielli, the rate is fixed at 10% after a preliminary analysis in which this rate "provided stable predictive models over several runs"

(Gabrielli, 2020b: 150, Section 3.3.2 of Paper C); Härkönen used 20% in the double network (Härkönen, 2021: 17).

3.5 SUMMARY

Three devices keep the bCCNN close to the chain ladder: the start at the chain ladder, the number of steps and dropout. The step-size bound of RMSprop limits every coordinate to a distance of $0.00316 t$ from its start and the correction to a size linear in t , so the number of steps is the control on the distance from the chain ladder. In the quadratic picture, t steps act as a ridge penalty of strength $1/(\eta t)$ towards the chain ladder: well-supported directions are learned and weakly supported ones, including the extrapolation corner, are barely moved. Dropout adds a further penalty at the cost of noisier validation curves. Chapter 4 turns from how the reserve is produced to how it is judged.

CHAPTER 4

EVALUATION OF THE RESERVE

4.1 INTRODUCTION

Chapters 2 and 3 described what the bCCNN estimates and why it stays near the chain ladder. This chapter describes how the resulting reserve is judged in the code and in the papers: the back-test against the simulated truth, the out-of-time test error of the rolling origin, the classical fits and the scoring functions of the repository, and the three diagnostic plots.

4.2 THE BACK-TEST AGAINST THE SIMULATED TRUTH

The function `bccnn_tables()` reports, for the chain ladder and the bCCNN, the reserve, the bias $\hat{R} - R$ and the percentage bias, the in-sample deviance on O , the out-of-sample deviance on F , and the dispersion (companion II, Section 5.2). The by-origin table splits the bias by accident period, and the script plots it. The four quantities are read together as follows.

In-sample deviance. It falls with t^* by construction (the row 'decrease of in-sample loss' of Table 3 of Paper C); a large decrease is not a merit, and it is the first symptom of over-fitting when the out-of-sample numbers do not follow.

Out-of-sample deviance on F . It is the cell-level criterion the model was trained for, evaluated where it was not trained; it penalises a model that obtains the right total by cancelling errors.

Bias. The bias is what the reserve is for. Table 2 of Paper C and Table 3 of Härkönen (2021: 33-34) are tables of percentage biases; the latter shows biases between 0.07 % and - 9 % across six LoBs for the simple network and sign changes when the halves are swapped, so a single bias on a single triangle is a weak basis for a verdict, and the by-origin biases and the out-of-sample deviance are needed to see whether the correction is systematic.

Payments after development period n . They are reported but modelled by neither model: the comparison is reserve to period n against truth to period n .

4.3 THE OUT-OF-TIME TEST ERROR

Under the rolling origin, the test error $\hat{D}$ of companion IV, (3.4), is the only evaluation available *before* the truth is used: the deviance per test cell of the whole procedure replayed at valuation years 15 and 18, next to the chain ladder at those dates. It is reported on one seed; Al-Mudafer *et al.* average it over several initialisations before comparing designs (Al-Mudafer *et al.*, 2021: 16).

4.4 THE CLASSICAL FITS AND THE SCORES IN R/SCORING

The repository fits the same triangle with `fit_chainladder()` (the chain-ladder factors, no standard error), `fit_mack()` (the model of Mack (1993): the same point estimate with a distribution-free standard error) and `fit_glm_reserve()` (`glmReserve()` with `var.power = 1`, the same ccODP as companion I, with a standard error by the delta method or by the bootstrap of England and Verrall (2002)). The script asserts that the ccODP of the bCCNN and the chain ladder agree on the reserve; the Mack and GLM fits therefore also share the *starting point* of the bCCNN, and differ from it only in having a standard error.

The function `score_total(reserve, se, truth, sims)` in R/scoring scores a total reserve by the error and the percentage error; the standardised error $z = (R - \hat{R})/\hat{se}$ and its normal probability integral transform (PIT); whether the truth lies in the 95% interval; the continuous ranked probability score (CRPS), from a normal predictive distribution with mean $\hat{R}$ and standard deviation $\hat{se}$,

$$CRPS(\hat{R}, \hat{se}; R) = \hat{se} \left(z(2\Phi(z) - 1) + 2\phi(z) - \frac{1}{\sqrt{\pi}} \right), \quad (4.1)$$

or from simulations when they are supplied; and the quantile scores at the 75th and 95th percentiles. As the code stands, the bCCNN has no $\hat{se}$ and no simulations, so only the first two of these are available for it. Chapter 5 is about supplying the rest.

4.5 THE THREE DIAGNOSTIC PLOTS

The relative difference $e^T - 1$ shows *where* the network moved the chain ladder and by how much, over the whole square. The cumulative development factors show the same thing as development patterns by accident period. The Pearson residuals on the lower triangle, with each model's own $\hat{\phi}$ and a common colour scale, show *whether* the move was in the right direction cell by cell; a pattern along the diagonals is the calendar-effect signature of Section 2.4, and a pattern in the latest accident periods is the extrapolation corner.

4.6 SUMMARY

The reserve of the bCCNN is judged by the back-test against the simulated truth, where the in-sample deviance is not a merit, the out-of-sample deviance checks the cell means and the bias checks the total; by the out-of-time test error of the rolling origin, the only evaluation available before the truth is used; and by three diagnostic plots that show where and whether the network moved the chain ladder in the right direction. The classical fits share the starting point of the bCCNN but carry a standard error, so the scoring functions of the repository can judge their predictive distributions; for the bCCNN only the error and the percentage error are available. Chapter 5 shows how the missing standard error can be obtained.

CHAPTER 5

UNCERTAINTY OF THE RESERVE

5.1 INTRODUCTION

Chapter 4 ended with the observation that the bCCNN has no standard error in the code, so that most of the scores of the repository cannot be computed for it. This chapter shows how the uncertainty of the bCCNN reserve can be measured. It decomposes the mean squared error of prediction, gives the process variance under the over-dispersed Poisson assumption, adapts the parametric bootstrap to the whole fitting procedure, treats the variance that comes from the seed, and shows how the pieces feed the scores.

5.2 THE DECOMPOSITION

For a reserve estimate $\hat{R}$ computed from the observed triangle, the conditional mean squared error of prediction (MSEP) is (England and Verrall, 2002; Härkönen, 2021: 24, equation (78))

$$MSEP(\hat{R}) = E[(R - \hat{R})^2 / D] = \underbrace{Var(R / D)}_{\text{process variance}} + \underbrace{(E[R / D] - \hat{R})^2}_{\text{estimation error}}, \quad (5.1)$$

the second term being replaced in practice by its expectation over the sampling of the triangle, the estimation variance (EV), $EV = Var(\hat{R})$. Model error, the possibility that neither the ccODP nor the bCCNN is the true mean structure, is outside (5.1); Härkönen makes the point in her discussion that "a model with large relative bias may still achieve low MSEP" (Härkönen, 2021: 52) for exactly this reason, so (5.1) is read next to the back-test bias, not instead of it.

5.3 PROCESS VARIANCE

Under the over-dispersed Poisson (ODP) assumption on the future cells, independent with $Var(Y_{i,j}) = \phi \mu_{i,j}$, the process variance (PV) is

$$\begin{aligned} PV &= Var(R / D) = \phi \sum_{(i,j) \in F} \mu_{i,j}, \\ \widehat{PV}^M &= \hat{\phi}^M \hat{R}^M, \end{aligned} \quad (5.2)$$

for each model M with its own dispersion: $\hat{\phi}_D$ for the chain ladder (companion I, (2.15)) and the reduced $\hat{\phi}^{bCCNN}$ of Paper C (companion I, (5.2)). Equation (80) of Härkönen (2021: 24) is (5.2) with claim counts as exposures. The process standard deviation $\sqrt{\phi \hat{R}}$ is the irreducible part: it is there even if the means were known exactly.

5.4 ESTIMATION VARIANCE: THE PARAMETRIC BOOTSTRAP

For the ccODP, `fit_glm_reserve()` gives EV by the delta method (`mse.method = "formula"`): $\widehat{EV} = \sum_F \sum_F \hat{\mu}_{i,j} \hat{\mu}_{k,l} \widehat{Cov}(\hat{\eta}_{i,j}, \hat{\eta}_{k,l})$; England and Verrall, 1999) or by the bootstrap ("bootstrap"). The bCCNN has no formula, but it has a procedure that can be rerun on pseudo-data, which is all a bootstrap needs. Härkönen (2021: 24-25), in her Section 2.4 and her equations (82) and (83), uses the parametric form: simulate pseudo-triangles from the fitted over-dispersed Poisson, using $Y/\phi \sim \text{Poisson}(\mu/\phi)$. Written for the bCCNN, the steps are:

- i) For $b=1, \dots, B$, draw a pseudo upper triangle $Y_{i,j}^{*b} = \hat{\phi} \cdot \text{Poisson}(\hat{\mu}_{i,j}/\hat{\phi})$ on O , from a generating model $(\hat{\mu}, \hat{\phi})$: the ccODP (the null hypothesis that the chain ladder is right) or the bCCNN itself.
- ii) Refit the whole procedure on Y^{*b} (the ccODP, the start, and exactly t^* steps with the configured seed, or a fresh seed per replicate, which folds the seed variance of Section 5.5 into the result) and read off $\hat{R}^{*b}$.
- iii) Compute $\widehat{EV} = \frac{1}{B-1} \sum_b (\hat{R}^{*b} - \bar{R}^*)^2$ and $\widehat{MSEP} = \widehat{PV} + \widehat{EV}$.

Keeping t^* fixed across replicates treats the number of steps as part of the model, as Paper C does when it uses one t^* for all LoBs; re-choosing t^* inside each replicate would also capture the variability of the early-stopping rule, at the cost of a validation run per replicate. The non-parametric alternative of England and Verrall (2002), resampling the scaled Pearson residuals of the ccODP to build pseudo-triangles, is what `glmReserve(mse.method = "bootstrap")` does and would serve the bCCNN equally. The file `config.yml` already carries `bootstrap: iterations.NN: 10`; each replicate costs one `glm()` and t^* full-batch steps, a few seconds. A full predictive distribution, for the PIT and CRPS of `score_total()`, adds a fourth step: draw the future cells $Y_{i,j}^{*b}, (i, j) \in F$, from $ODP(\mu_{i,j}^{*b}, \hat{\phi})$ and record R^{*b} as a simulated outcome.

5.5 SEED VARIANCE AND THE NAGGING PREDICTOR

A network has a source of estimation variance a GLM does not have: the seed. Härkönen refitted the double network for 20 seeds at the chosen number of epochs and found biases spread over several percentage points, concluding that "it might be beneficial to fit several models with different seeds and then choose the average prediction" (Härkönen, 2021: 37), a recommendation she repeats in her discussion (Härkönen, 2021: 53); Al-Mudafer *et al.* ensemble five fits (Al-Mudafer *et al.*, 2021: 17). Richman and Wüthrich (2020) call the average over seeds the nagging predictor,

$$\begin{aligned}\hat{R}_K &= \frac{1}{K} \sum_{k=1}^K \hat{R}^{(s_k)}, \\ \text{Var}(\hat{R}_K) &\approx \frac{\sigma_s^2}{K} + (\text{variance common to all seeds}),\end{aligned}\tag{5.3}$$

where σ_s^2 is the across-seed variance of a single fit. Averaging removes the seed-specific part of the error at rate $1/K$ and leaves the part that comes from the data, which is the estimation variance of Section 5.4 proper. In the bCCNN the seed enters only through the hidden-layer initialisation and the dropout masks (companion II, Section 3.4); the start and the ccODP are the same for every seed, so σ_s measures how much the *correction* Γ depends on the random features it was built from. A seed study is a loop over seed in `bccnn_calibrate()` and a mean of the reserves; the neural network chain-ladder strand of the repository already does this with 20 seeds.

5.6 PUTTING THE NUMBERS TOGETHER

With $\widehat{se} = \sqrt{MSEP}$ from (5.1) and (5.2) and the bootstrap, `score_total()` gives the bCCNN the same report as the classical fits: the standardised error z , whose magnitude says whether the back-test bias is within the uncertainty of the model itself; the interval coverage; and the CRPS (4.1), which rewards a reserve that is both close to the truth and honest about its uncertainty. For a reserve that is a sum of means, the right comparison across models is then no longer ‘whose bias is smaller’ but ‘whose predictive distribution scores better’, which is where the dispersion rule of companion I, Section 5.3, with its $\max(0, \cdot)$ cut, becomes consequential: a bCCNN whose validation decrease was large is given a *smaller* process variance than the chain ladder, and that is a claim the back-test should be allowed to contradict.

5.7 SUMMARY

The uncertainty of the bCCNN reserve can be measured with the same decomposition as for the classical models: the MSE is the process variance plus the estimation variance, with model error outside both. The process variance follows from the ODP assumption and the dispersion of each model; the estimation variance can be obtained by a parametric bootstrap that reruns the whole procedure with a fixed number of steps; and the variance that comes from the seed can be measured by a seed study and reduced by the nagging predictor. With a standard error in hand, the scoring functions of the repository give the bCCNN the same predictive scores as the classical fits, and the dispersion rule of Paper C becomes testable. Chapter 6 places the bCCNN among the other models and evaluates these quantities on a synthetic run.

CHAPTER 6

THE bCCNN IN CONTEXT AND A NUMERICAL ILLUSTRATION

6.1 INTRODUCTION

Chapters 2 to 5 developed what the bCCNN estimates, why it is regularised, how it is judged and how its uncertainty can be measured. This chapter first places the bCCNN among the reserving models of the thesis, then shows where each formula and step of this report is located in the code, and finally evaluates the quantities of Chapters 3 and 5 on a synthetic run. It closes with the summary and conclusions of the report.

6.2 THE BCCNN AMONG THE MODELS OF THE THESIS

Table 6.1 compares the mean structure, the fitting method, the uncertainty measure and the location in the repository of five models. The GLM form of the ccODP is fitted by quasi-Poisson iteratively weighted least squares (IWLS). The first three rows share the same point of departure, the chain-ladder reserve.

Table 6.1 Where the bCCNN sits among the models of the thesis

Model	Mean structure	Fitted by	Uncertainty	In the repository
Chain ladder and Mack	$\hat{C}_{i,n} = C_{i,n+1-i} \prod f_k$	Volume-weighted factors	Distribution-free standard error of Mack (process and estimation)	<code>fit_chainladder()</code> , <code>fit_mack()</code>
ccODP GLM	$\exp\{c + \alpha_i + \beta_j\}$, same reserve as the chain ladder	Quasi-Poisson IWLS	Delta method or bootstrap (England and Verrall, 2002)	<code>fit_glm_reserve()</code> , <code>fit_odp_glm()</code>
bCCNN (Paper C)	ccODP $\times e^{\Gamma}$, Γ a network on $(\hat{\alpha}_i, \hat{\beta}_j)$, started at 0	t^* RMSprop steps, early-stopped	None in the code; Chapter 5	<code>bccnn_calibrate()</code>
Double network (Gabrielli, 2020a; Härkönen, 2021: 16)	Counts and amounts jointly, with reporting and payment delay, attention over past counts	The same kind of procedure, scaled losses	Bootstrap (Härkönen, 2021: 24-25)	Not in this repository
Neural network chain ladder (Wüthrich, 2018)	Factors $f_{j-1}(x)$ depending on claim features	One network per development period, chain-ladder start	Nagging over seeds	The R/nn_chain_ladder. R strand

6.3 WHERE THE PIECES ARE IN THE CODE

Table 6.2 maps each formula and step of this report to the functions, settings and outputs of the code; where a step is not yet implemented, the table says so.

Table 6.2 Where the formulas and steps of this report are in the code

Topic	Where
The correction I^r and the two reserves, (1.1) and (1.2)	<code>bccnn_model()</code> , <code>fit_bccnn()</code> , <code>reserve_by_origin()</code> ; the heat map <code>plot_relative_difference(res\$odp\$mu, res\$nn\$mu)</code>
Deviance as the criterion, Section 2.2	<code>compile(loss = "poisson")</code> ; <code>poisson_deviance()</code> ; the <code>vali</code> , <code>test</code> , <code>truth</code> columns
Balance, (2.2)	Not in the code; <code>sum(tabs\$mu_bCCNN[upper cells])</code> against <code>sum(sets\$upper)</code> shows the gap
The budget, (3.1) and (3.2)	<code>optimizer_rmsprop(learning_rate = 0.001, rho = 0.9)</code> , <code>max_epochs</code> , <code>epochs</code> in <code>config.yml</code>
Dropout, Section 3.4	<code>layer_dropout(rate = 0.1)</code> after each hidden layer
The back-test, Section 4.2	<code>bccnn_tables()</code> : rows "bias", "in-sample loss", "out-of-sample loss", "payments after the last development period"; by <code>origin</code> ; <code>plot_bias_by_origin()</code>
The out-of-time test error, Section 4.3	<code>bccnn_rolling_origin()</code> <code>\$test_error</code> ; <code>bccnn_tables()</code> <code>\$rolling_origin</code>
Classical fits and scores, Section 4.4	<code>fit_chainladder()</code> , <code>fit_mack()</code> , <code>fit_glm_reserve()</code> in <code>R/classical_models.R</code> ; <code>score_total()</code> , <code>crps_norm()</code> , <code>qscore()</code> in <code>R/scoring</code>
Process variance, (5.2)	$\hat{\phi}$ from <code>res\$phi</code> , reserves from <code>res\$odp\$reserve</code> , <code>res\$nn\$reserve</code> ; not yet multiplied in the code
Bootstrap, Section 5.4	Not yet in the code for the bCCNN; <code>fit_glm_reserve(mse.method = "bootstrap")</code> for the ccODP; <code>config.yml</code> <code>bootstrap</code> : <code>iterations.NN</code>
Seed study, Section 5.5	A loop over <code>seed =</code> in <code>bccnn_calibrate()</code> ; <code>cfg\$seed</code>

6.4 NUMERICAL ILLUSTRATION

The numbers below come from the same synthetic run as companion II, Section 6.3: a 20×20 triangle with a mild interaction the chain ladder cannot represent, $t^* = 407$ chosen on the claims split, and a refit on the full triangle. They illustrate the formulas of this report; they say nothing about the simulated portfolio of the thesis.

Figure 6.1 shows two heat maps. The left panel is the learned correction $e^I - 1$ of the synthetic refit over the whole square; the line is the latest diagonal, and the grey columns are development periods without payments, where the ccODP effect is $-\infty$ and the ratio is meaningless. The right panel is the interaction the synthetic data were built with, $e^{v^{(i-10.5)|j-4.5)}1_{\{j \leq 12\}} - 1$, the multiplicative departure from a cross-classified structure. After 407 steps the correction is within a few per cent, varies mostly with the development period and only mildly with the accident period, and is carried smoothly into the lower triangle: the network has moved a little way from the chain ladder, in the direction the data pull, and has not reproduced the designed interaction, whose strongest cells (late accident periods at development periods 8 to 12) lie in or near the extrapolation corner.

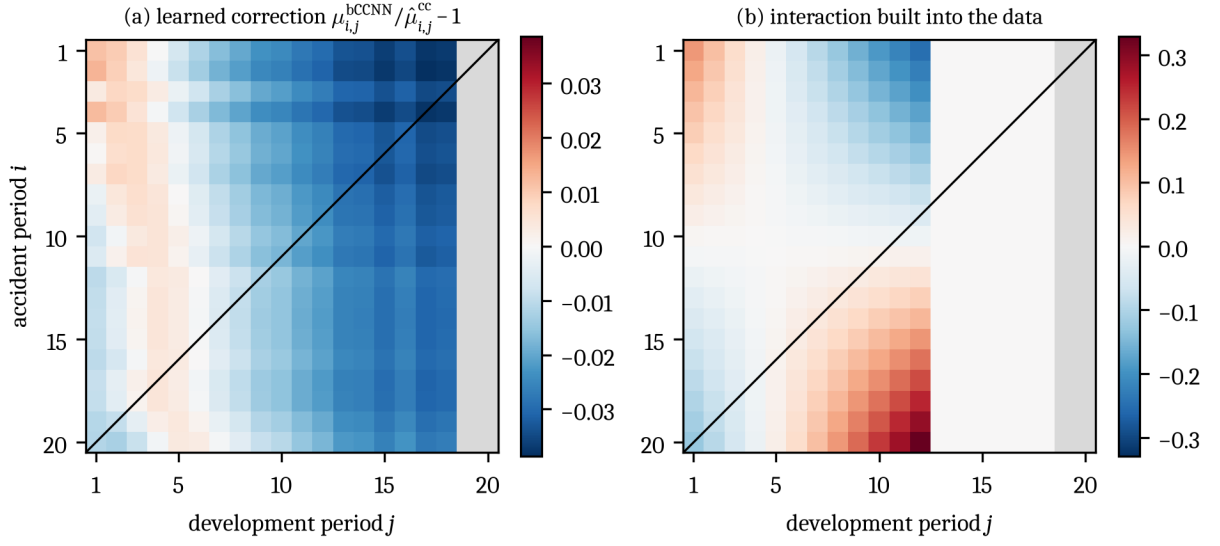


Figure 6.1 The learned correction of the synthetic refit and the interaction built into the synthetic data

Table 6.3 lists the quantities of Chapters 3 and 5 on this run.

Table 6.3 Quantities of early stopping and uncertainty on the synthetic run (amounts in millions of the synthetic data)

Quantity	Value
Steps t^* ; bound (3.1) on any coordinate, $0.00316 t^*$	407; 1.29
Realised distance from the start, $\max_{\theta} \hat{\vartheta} - \vartheta_0 $: over all 547 parameters; over the output weights B ; w ; c	0.313 (a hidden-layer weight); 0.026; 0.008; 0.003
Realised correction, $\max_{i,j} \Gamma_{\hat{\theta}}(i, j) $: over the development periods with payments; over their future cells	0.039; 0.038
Reserves: chain ladder, bCCNN, truth	1 227.3; 1 218.9; 1 438.8
Dispersions: $\hat{\phi}_D, \hat{\phi}^{bCCNN}$	0.314; 0.307
Process standard deviation $\sqrt{\hat{\phi} \hat{R}}$, (5.2): chain ladder, bCCNN	19.6; 19.3
Parametric bootstrap, $B = 20$ pseudo-triangles from the ccODP, $t^* = 407$ fixed, same seed: estimation standard deviation of the chain-ladder reserve, of the bCCNN reserve; correlation of the two	26.1; 26.1; 1.00
$\sqrt{MSEP}$, (5.1): chain ladder, bCCNN	32.6; 32.4
Seed study, $K = 10$ seeds, $t^* = 407$: mean, standard deviation and range of the bCCNN reserve	1 231.0; 7.7; 1 221.0 to 1 242.1
Nagging predictor $\hat{R}_{10}$, (5.3)	1 231.0

Four readings follow from Table 6.3. First, the realised distance from the start is a quarter of the bound (3.1) for the most-moved coordinate and far less for the output weights that carry the correction: the

gradients change sign, and the network settles much closer to the chain ladder than the budget allows, which is the shrinkage picture of Section 3.3.

Second, the bootstrap, which generates pseudo-triangles from the chain ladder, finds the estimation standard deviation of the bCCNN reserve equal to that of the chain ladder to three digits and the two reserves correlated at 0.998 across pseudo-triangles: when the chain ladder is the truth, the correction learned in 407 steps is noise that follows the chain ladder of each pseudo-triangle, and it adds no variance of its own at a fixed seed.

Third, the variance the network does add is the seed spread, 7.7 on a reserve of about 1 230 (0.6%), about a third of the bootstrap standard deviation; the nagging average over ten seeds, 1 231.0, removes most of it.

Fourth, the biases of both models on this synthetic triangle, -211 and -220 , are six to seven times their $\sqrt{MSEP}$ of about 32: the interaction grows into the lower triangle, and neither the chain ladder nor a correction that must extrapolate from $(\hat{\alpha}_i, \hat{\beta}_j)$ pairs it has not seen can know that. That is model error, outside (5.1), and the back-test bias is the only number that sees it.

6.5 SUMMARY AND CONCLUSIONS

This report has joined the companion explainers into one account of the bCCNN. The model is the chain ladder multiplied by a correction e^r that is zero at the start, a smooth function of the two ccODP effects, and learned in t^* RMSprop steps. It estimates the mean of each cell, a target for which the Poisson deviance is strictly consistent, and the reserve is the sum of these means over the future cells; unlike the chain ladder it does not reproduce the observed total, which the rescaling (2.2) would restore.

The model does not over-fit its 210 cells because the step-size bound of RMSprop limits the distance from the chain ladder linearly in the number of steps, because early stopping acts approximately as a ridge penalty of strength $1/(\eta t)$ towards the chain ladder, and because dropout penalises co-adapted features. In the synthetic run the parameters moved a quarter of the bound at most and the correction stayed within a few per cent.

The reserve is judged by the back-test bias, the out-of-sample deviance, the out-of-time test error and three diagnostic plots. The scores that need a predictive distribution are not yet available for the bCCNN, because the code computes no standard error for it. The MSEP can be supplied by the process variance under the ODP assumption, a parametric bootstrap of the whole procedure and a seed study, at a cost of a few seconds per replicate. On the synthetic run the bootstrap estimation variance of the bCCNN equals that of the chain ladder, the seed adds a spread of about a third of the bootstrap standard deviation that the nagging predictor largely removes, and both models carry a bias many times their standard error, a model error that only the back-test reveals.

Three extensions of the code follow from these conclusions: a parametric bootstrap and a seed study for the bCCNN, so that `score_total()` can give it the same report as the classical fits; a decision on whether to apply the balance correction (2.2); and the use of the predictive scores to test the claim, implied by the dispersion rule of Paper C, that a bCCNN with a large validation decrease has a smaller process variance than the chain ladder.

References

- Al-Mudafer, M.T., Avanzi, B., Taylor, G. & Wong, B. 2021. Stochastic loss reserving with mixture density neural networks. arXiv preprint 2108.07924.
- Bishop, C.M. 1995. Regularization and complexity control in feed-forward networks. *Proceedings of the International Conference on Artificial Neural Networks 1995*. 141-148.
- Department of Statistics and Actuarial Science. 2026. *Research assignment guide*. Last updated 28 January 2026. Stellenbosch: Stellenbosch University.
- England, P.D. & Verrall, R.J. 1999. Analytic and bootstrap estimates of prediction errors in claims reserving. *Insurance: Mathematics and Economics*, 25(3):281-293.
- England, P.D. & Verrall, R.J. 2002. Stochastic claims reserving in general insurance. *British Actuarial Journal*, 8(3):443-518.
- Gabrielli, A. 2020a. A neural network boosted double over-dispersed Poisson claims reserving model. *ASTIN Bulletin*, 50(1):25-60.
- Gabrielli, A. 2020b. Claims reserving and neural networks. Unpublished doctoral thesis. Zurich: ETH Zurich.
- Gabrielli, A., Richman, R. & Wüthrich, M.V. 2020. Neural network embedding of the over-dispersed Poisson reserving model. *Scandinavian Actuarial Journal*, 2020(1):1-29.
- Gabrielli, A., Richman, R. & Wüthrich, M.V. 2018. Neural network embedding of the over-dispersed Poisson reserving model. SSRN working paper 3288454, version of 21 November 2018.
- Gneiting, T. 2011. Making and evaluating point forecasts. *Journal of the American Statistical Association*, 106(494):746-762.
- Goodfellow, I., Bengio, Y. & Courville, A. 2016. *Deep learning*. Cambridge, MA: MIT Press.
- Härkönen, V. 2021. On claims reserving with machine learning techniques. Unpublished master's thesis. Stockholm: Stockholm University.
- Mack, T. 1993. Distribution-free calculation of the standard error of chain ladder reserve estimates. *ASTIN Bulletin*, 23(2):213-225.
- Richman, R. & Wüthrich, M.V. 2020. Nagging predictors. *Risks*, 8(3):83.
- Wüthrich, M.V. 2020. Bias regularization in neural network models for general insurance pricing. *European Actuarial Journal*, 10:179-202.
- Wüthrich, M.V. & Merz, M. 2019. Editorial: Yes, we CANN! *ASTIN Bulletin*, 49(1):1-3.

Wüthrich, M.V. & Merz, M. 2023. *Statistical foundations of actuarial learning and its applications*. Cham: Springer.

Wüthrich, M.V. 2018. Neural networks applied to chain-ladder reserving. *European Actuarial Journal*, 8:407-436.

APPENDIX A

LIST OF SYMBOLS

The symbols are those of companions I, II and IV, as listed in the upper block of the table; the lower block lists the symbols that are new in this report, with the section or chapter in which each is introduced.

Table A.1 Symbols used in this report

Symbol	Meaning	Section
n, i, j, k	Triangle size 20; accident, development and calendar period	1.2
O, F	Observed cells (210) and future cells (190)	1.2
$Y_{i,j}, y_{i,j}; R$	Payments of a cell; the true outstanding payments $\sum_F Y_{i,j}$	1.2
$\hat{\mu}_{i,j}^{cc}, \hat{\phi}_D$	Fitted ccODP means and dispersion (companion I, (2.1) and (2.15))	1.2
$\mu_{i,j}(\theta), \theta_0, t^*, \hat{\theta}$	The means of the network, its start, the number of steps, the fitted parameters	1.2
$\hat{R}^{cc}, \hat{R}^{bCCNN}$	The two reserves, sums of the means over F	1.2
$D(y, \mu; S)$	Poisson deviance on the cells S	2
η, ρ	RMSprop learning rate and decay, 0.001 and 0.9	3
	Symbols new in this report	
$\Gamma_\theta(i, j)$	The log-scale correction of the chain ladder by the network, $\log \mu_{i,j}(\theta) - \log \hat{\mu}_{i,j}^{cc}$	1.2
$d(y, \mu)$	One summand of the deviance, $\mu - y + y \log(y/\mu)$	2.2
H, λ_r	Hessian of the loss at its minimiser and its eigenvalues (quadratic picture of early stopping)	3.3
$MSEP$	Conditional mean squared error of prediction of a reserve	5
PV, EV	Process variance and estimation variance	5
$Y^{*b}, \hat{R}^{*b}$	A bootstrap pseudo-triangle and the reserve refitted on it, $b = 1, \dots, B$	5.4
$\hat{R}^{(s)}, \hat{R}_K$	The reserve for seed s ; the average over K seeds (nagging predictor)	5.5
z	Standardised reserve error $(R - \hat{R})/\widehat{se}$	5.6